C programming

Trainer : Rajiv K

Email : rajiv.kamune@sunbeaminfo.com

Agenda

Dynamic Array
- **malloc, calloc, realloc**

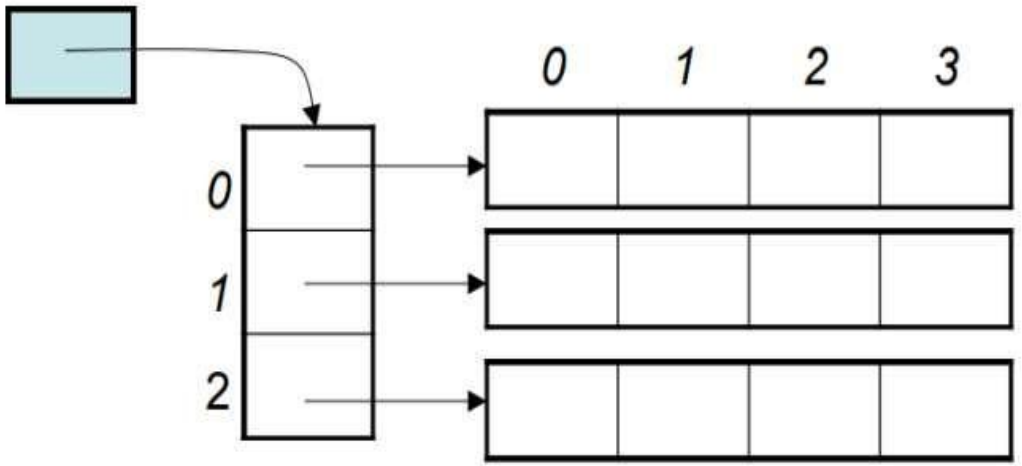
Memory Leakage

Dangling Pointer

Pre-processor Directives
- text here

2-D arrays

- Example: int a[3][5];
  - Logically it may be viewed as a two-dimensional collection of data, three rows and five columns, each location is of type int.



	0	1	2	3	4
0	a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]
1	a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[1][4]
2	a[2][0]	a[2][1]	a[2][2]	a[2][3]	a[2][4]

- **A 2D array is a 1D array of (references to) 1D arrays.**

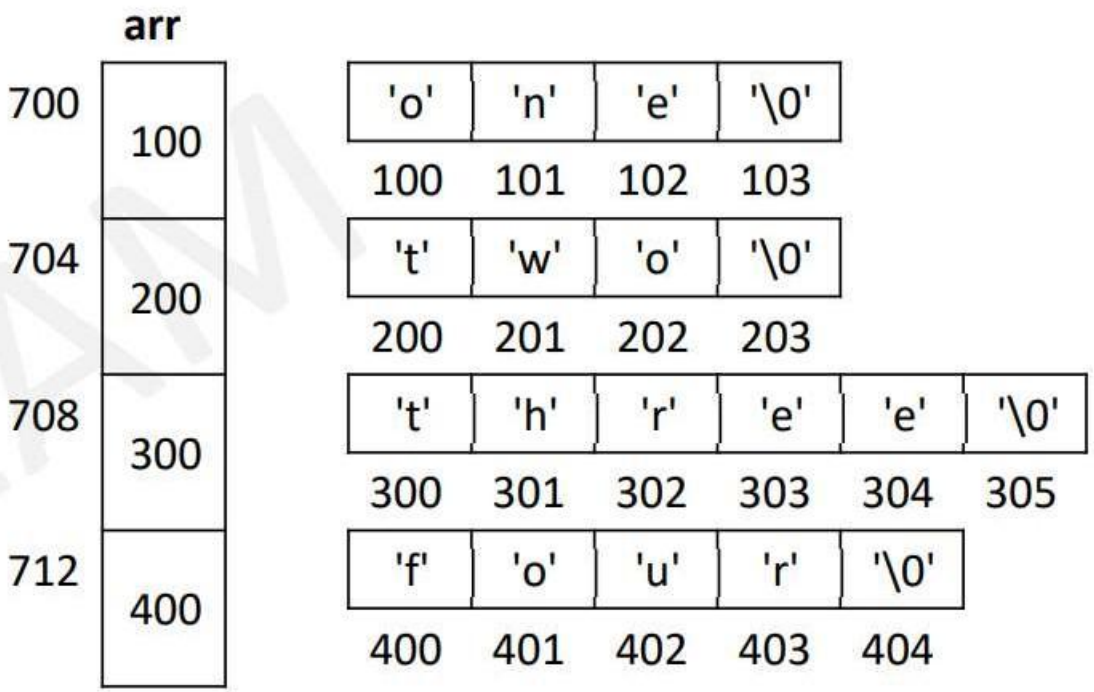
2-D Arrays :

- 2-D array is collection of 1-D arrays in contiguous memory locations.
  - Each element is 1-D array.
- int arr[3][4] = { {1, 2, 3, 4}, {10, 20, 30, 40}, {11, 22, 33, 44} };

	0				1				2			
	0	1	2	3	0	1	2	3	0	1	2	4
arr	**1**	**2**	**3**	**4**	**10**	**20**	**30**	**40**	**11**	**22**	**33**	**44**
	400	404	408	412	416	420	424	428	436	440	444	448
	400				416				436			

Array of pointers :

```
char *arr[] = { "one", "two", "three", "four" };
for(i = 0; i < 4; i++)
        puts(arr[i]);
```



Passing 2-D array to functions :

- 2-D array is passed to function by address.
- It can be collected in formal argument using array notation or pointer notation.
- While using array notation, giving number of rows is optional. Even though mentioned, will be ignored by compiler.

Dynamic Memory Allocation :

- Dynamic memory allocation allow **allocation** of **memory at runtime** as per requirement.
- This memory is allocated at runtime on **Heap section of process**.
- Library functions used for Dynamic memory allocation are
- malloc() – allocated memory contains **garbage values.**
- calloc() – allocated memory contains **zero values.**
- realloc() – allocated memory **block** can be **resized (grow or shrink).**

- All these function **returns base address** of allocated block as void*.
- If function **fails**, it returns **NULL** pointer.
- The memory block allocated by malloc() has a garbage value.
- The memory block allocated by calloc() is initialized by zero.

Dynamic Memory Allocation :

- Dynamic memory can be requested using malloc,calloc,realloc function
- Such requested memory will be in **control of user programmer.**
- On demand programmer request memory utilse same and once job is finished programmer has to release such dynamic memory.
- We can **shrink or grow dynamic** memory at runtime.
- malloc,calloc,realloc function provides memory from **heap section.**
- If request is successful they will return base address of memory else return NULL.
- void* malloc(int size);
- void* calloc(int count,int ele size);
- void* realloc(void *mem block,int size);
- The address returned by these functions should be type-casted to the required pointer type.

Difference between malloc() and calloc() :

S.No.	malloc()	calloc()
1.	malloc() function creates a single block of memory of a specific size.	calloc() function assigns multiple blocks of memory to a single variable.
2.	The number of arguments in malloc() is 1.	The number of arguments in calloc() is 2.
3.	malloc() is faster.	calloc() is slower.
4.	malloc() has high time efficiency.	calloc() has low time efficiency.
5.	The memory block allocated by malloc() has a garbage value.	The memory block allocated by calloc() is initialized by zero.
6.	malloc() indicates memory allocation.	calloc() indicates contiguous allocation.

Memory Leakage :

- If memory is allocated dynamically, **but not released is said to be "memory leakage".**
- **Such memory is not used by OS** or **any other application** as well, so it is wasted.
- In modern OS, leaked memory gets auto released when program is terminated.
- However for long running programs (like web-servers) this memory is not freed.
- More memory leakage reduce available memory size in the system, and thus slow down whole system.
- In Linux, valgrind tool can be used to detect memory leakage.
- int main() {

```
int *p = (int*) malloc(20);
int a = 10;
p = &a; // here addr of allocated block is lost, so this memory can never be freed.
// this is memory leakage
return 0;
}
```

Dangling Pointer :

- Pointer keeping address of memory that is not valid for the application, is said to be "dangling pointer".
- Any read/write operation on this may abort the application. In Linux it is referred as "Segmentation Fault".
- Examples of dangling pointers
  - After releasing dynamically allocated memory, pointer still keeping the old address.
  - Uninitialized (local) pointer
  - Pointer holding address of local variable returned from the function.

It is advised to assign NULL to the pointer instead of keeping it dangling.

```
int main() {
int *p = (int*) malloc(20);
free(p); // now p become dangling
return 0;
}
```

Preprocessor Directives :

- Preprocessor is part of C programming toolchain/SDK.
  - Removes comments from the source code.
  - Expand source code by processing all statements starting with #.
  - Executed before compiler
- All statements starting with # are called as preprocessor directives.
  - Header file include
    - #include
  - Symbolic constants & Macros
    - #define
  - Conditional compilation
    - #if, #else, #elif, #endif
    - #ifdef #ifndef
  - Miscellaneous
    - #pragma, #error

Preprocessor Directives :

- Preprocessor is small application helps to generate intermediate code.
- It process all c statements starting with # which is called as preprocessor directives.
- Preprocessor directives are the commands given preprocessor about processing source code before compilation.

- can be used for defining symbolic constant or macro
- #define <symbol> <replacable text>

#include :

- #include includes header files (.h) in the source code (.c).
- #include <file.h>
  - Find file in standard include directory.
  - If not found, raise error.
- #include "file.h"
  - File file in current source directory.
  - If not found, find file in standard include directory.
  - If not found, raise error.

#define symbolic constants :

- Used to define symbolic constants.
  - #define PI 3.142
  - #define SIZE 10
- Predefined constants
  - __LINE__
  - __FILE__
  - __DATE__
  - __TIME__
- Symbolic constants and macros are available from there declaration till the end of file. Their scope is not limited to the function.

#define macros :

- Used to define macros (with or without arguments)
  - #define ADD(a, b) (a + b)
  - #define SQUARE(x) ((x) * (x))
  - #define SWAP(a,b,type) { type t = a; a = b; b = t; }
- Macros are replaced with macro expansion by preprocessor directly.
  - May raise logical/compiler errors if not used parenthesis properly.
- Stringizing operator (#)
  - Converts given argument into string.
  - #define PRINT(var) printf(#var " = %d", var)
- Token pasting operator (##)
  - Combines argument(s) of macro with some symbol.
  - #define VAR(a,b) a##b

Difference between functions and macros :

- Functions
  - Function have declaration, definition and call.
  - Functions are called at runtime by creating FAR on stack.
  - Functions are type-safe.
  - Functions may be recursive.
  - Functions called multiple times doesn't increase code size.
  - Functions execute slower.
  - For bigger reusable code snippets, functions are preferred.
- Macros
  - Macro definition contain macro arguments and expansion.
  - Macros are replaced blindly by the processor before compilation
  - Macros are not type-safe.
  - Macros cannot be recursive.
  - Macros (multi-line) called multiple times increase code size.
  - Macros execute faster.
  - For smaller code snippets/formulas, macros are preferred.

Conditional Compilation :

- As preprocessing is done before compilation, it can be used to control the source code to be made available for compilation process.
- The condition should be evaluated at preprocessing time (constant values).
- Conditional compilation directives
  - #if, #elif, #else, #endif
  - #ifdef, #ifndef

- #if
- #else
- #elif
- #endif
- #ifdef
- #ifndef
- Example:
- #define PI 3.14 ,
- #if defined (PI)
- printf("DEFINED");
- #else
- printf("NOT DEFINED");
- #endif

Thank you!!